

C A S E S T U D Y · 2 0 2 6

MitchAI

Hospitality software prototype case study

Status & implementation boundary — What this case study does and does not claim

Status: MitchAI is a technical portfolio prototype, not a public-facing production service.

Repository: <https://github.com/Bogdan0708/MitchAI> An n8n master orchestrator (8 sanitised workflows in automations/n8n) has synthetic execution evidence, and a historical app deployment to AWS ECS Fargate + RDS is evidenced by a GitHub Actions workflow (green run 2026-02-12, image build + ECS service update); Terraform modules for GCP and AWS are published separately and are not shown to have provisioned that run. Neither is offered as a hosted product with an SLA, and no independently assessed GDPR/compliance status is claimed.

Role: Founder and engineer, drawing on 10+ years of hospitality operations.

Author: Vasile Bogdan Godja · godjabogdan@gmail.com · github.com/Bogdan0708

01 Product scope

Hospitality workflows represented in code

Checked-in scope

- Routes and services for menus, orders, reservations, reviews, content, analytics and AI-assisted tasks.
- n8n master orchestrator routing to Menu / Social / Strategy / Feedback / Operations workflows (automations/n8n, 8 sanitised workflows).
- HACCP recordkeeping module; WhatsApp Cloud API and Telegram integration; PostgreSQL row-level security policies.
- Authentication, tenant context, validation, rate limiting and PostgreSQL schema/migrations; a Next.js dashboard for hospitality and AI workflow surfaces.
- Multi-provider LLM orchestrator, including local Ollama and LM Studio alongside cloud providers.

Boundary

Modules may require credentials, external services and further integration testing. They are implementation evidence, not evidence that every workflow runs end to end for customers.

02 Architecture

Primary runtime stated accurately

Primary stack

The backend is Node.js with TypeScript and Express. The frontend is Next.js/React, and PostgreSQL with row-level security policies is the primary persistence layer. Redis and Qdrant are supporting services.

Eight published n8n workflows run through 10 synthetic HTTP scenarios in an isolated fixture harness, including a simulated provider failure. Provider endpoints and the database sink are substituted. This does not establish production execution; Ollama and LM Studio are the local-model options inside the multi-provider orchestrator. Python and FastAPI are not the primary MitchAI runtime.

A historical GitHub Actions run (2026-02-12) built an image and updated AWS ECS. Current production health is unverified. Published Terraform modules are separate implementation evidence; they are not shown to have provisioned that run.

Engineering posture

- Human approval remains the intended boundary for posts, messages, filings and purchases.
- Security-oriented controls (row-level security policies, HACCP module) are code-level signals, not certification or a live-service guarantee.

03 Evidence & next steps

What the prototype demonstrates

What it proves

MitchAI demonstrates a broad hospitality-domain TypeScript/Express backend, a published n8n orchestration layer, a Next.js frontend, a row-level-secured PostgreSQL data model, a multi-provider AI orchestrator, a GitHub Actions-driven AWS ECS Fargate + RDS deployment (green run 2026-02-12; Terraform modules published separately, not shown to have provisioned that run), and 193 passing backend tests (none skipped). Public-history cleanup and current production verification remain pending.

Remaining gates

- Validate real-provider credentials and database persistence; combo currently returns a plan without executing its steps.
- Retain dated test/build and per-node fixture evidence; resolve public-history findings and review fresh scanner results.
- Complete security, privacy and deployment review before production or compliance language.